

# SUMO 생성기 v3 시나리오 → 학습 공장 투입 결과

현준의 부탁 ①②③ 처리 보고 · 2026-08-18 · 최민서 · GitHub main 커밋 f4df259

## 한 줄 요약

- 부탁 ① (SUMO 투입 → 트리): 적시경보 규칙 55.3% → 모델 61.7% (같은 오경보 2.04%). 단, SUMO 실차에선 +15p 좋아졌지만 RC 스케일에선 -29p 나빠진 **실차 쏠림** — 그대로 실기 배포 불가.
- 부탁 ② (transformer): 전체 56.6% → 54.6%로 규칙 미달, 트리와 같은 쏠림. ONNX는 만들어 뒀음. 모델 종류보다 데이터 배합(SUMO:RC 위험 시나리오 4:1)이 먼저 고쳐야 할 것.
- 부탁 ③ (GPS 잡음 옵션): 완료. build\_dataset\_v5.py --gps-noise 로 켜,  $\sigma$  2.5 m 검증됨.
- 팀 코드 버그 1건 발견 (안 고침, 현준 판단): --streams와 --from-scenario-sim을 같이 주면 학습 단계에서 str/int id 정렬로 죽음. 우회 스크립트 제공.
- 추가 (현준 지시 (a) 실행): SUMO를 8:2로 줄이니 **RC에서 규칙을 이김 (58.0 → 61.7%)**, SUMO도 +21p 유지. risk\_model\_streams\_82.json = 배포 후보. → 3장

## 1. 부탁 ① — SUMO 시나리오를 학습 공장에 투입 (트리)

투입 재료: SUMO 생성기 v3 시나리오 700개(scenario\_9001~9700, 진짜 교내 주행·실차 5~25 m/s) × 시나리오당 최근접 2쌍 = 1,401 스트림 + 현준 명령 그대로 --from-scenario-sim 4000 --families --randomize.

학습셋: 3,411,227행 / 5,401 시나리오 (SUMO 1,635,367행·1,401개, scenario\_sim 1,775,860행·4,000개). 위험 시나리오: SUMO 1,099 vs scenario\_sim 274.

### 결과 (같은 분할·같은 임계값 0.9018, 평가 1,620 시나리오)

| 평가 범위                 | 위험 시나리오 | 적시경보 규칙 → 모델           | 오경보 규칙 / 모델   |
|-----------------------|---------|------------------------|---------------|
| 전체 (요청하신 숫자)          | 412     | 55.3% → 61.7% (+6.4p)  | 2.04% / 2.04% |
| SUMO (실차 5~25 m/s)    | 330     | 53.9% → 69.1% (+15.2p) | 1.92% / 3.50% |
| scenario_sim (RC 스케일) | 82      | 61.0% → 31.7% (-29.3p) | 2.15% / 0.74% |

주의 — 전체 숫자만 보면 안 됩니다. +6.4p는 실차 쪽으로 쏠린 평균입니다. SUMO 위험 시나리오가 RC의 4배라 트리가 실차 속도대를 우선 배웠고, RC 스케일(실기 환경)에서는 규칙보다 크게 나빠졌습니다. 이 risk\_model\_streams.json은 젓슨 실기에 그대로 올리면 안 됩니다.

긍정적인 부분: SUMO만 보면 경보 AI가 실차 속도를 처음 배운 효과가 뚜렷합니다(+15p). 배합만 맞추면 살릴 수 있는 재료입니다.

## 2. 부탁 ② — transformer (최근 10프레임) 학습

요청 규칙 그대로: 입력은 CSV의 FEATURE\_COLUMNS 15열(다른 특징 안 만들), 정답 y\_train, 채점은 시나리오 단위 timely\_alarm\_rate, 규칙과 같은 오경보율(2.17%)에서 임계값. 구조는 risk\_transformer\_v3와 같은 뼈대(d\_model 64.2층·68k 파라미터), 시퀀스는 시나리오 안에서만, 앞 9프레임은 첫 프레임 반복 패딩.

| 평가 범위        | 적시경보 규칙 → transformer |
|--------------|-----------------------|
| 전체           | 56.6% → 54.6% (규칙 미달) |
| SUMO         | 54.0% → 60.9%         |
| scenario_sim | 65.6% → 32.2%         |
| 정지 오경보       | 0.16% → 0.14%         |

해석: 트리를 못 이겼지만 첫 시도(6 epoch, 하이퍼파라미터 기본값)라 결론짓기는 이릅니다. 다만 실차 쓸림이 모델 종류와 무관하게 똑같이 나타난 것이 핵심 — 데이터 배합을 먼저 고친 뒤 재학습이 순서입니다.  
 산출물: models\_alarm\_tf/alarm\_transformer.onnx (입력 [batch,10,15], 출력 logit, ONNX 오차 1.5e-7) + alarm\_transformer\_scaler.json (mean/scale/threshold/채점 결과 메타). 젓슨 onnxruntime에서 바로 돌아가지만, 위 이유로 지금 배포는 권하지 않습니다.

### 3. (a) SUMO 축소 8:2 배합 → RC에서 규칙 이김 (현준 지시 후 추가 실행)

현준 지시대로 (a)만 먼저: sumo\_batch\_to\_streams.py --pairs 1 --limit 90 (SUMO 위험 75) + scenario\_sim 4,000 (RC 위험 274) RC 8 : SUMO 2. training\_dataset\_streams\_82.csv (1,878,375행 / 4,090 시나리오) → risk\_model\_streams\_82.json (임계값 0.8380, 규칙 오경보 2.18% 기준).

| 평가 범위                 | 위험 시나리오 | 적시경보 규칙 → 모델           | 오경보 규칙 / 모델   | 이전 (4:1 SUMO 우위) |
|-----------------------|---------|------------------------|---------------|------------------|
| 전체                    | 105     | 55.2% → 62.9% (+7.7p)  | 2.18% / 2.18% | 55.3 → 61.7%     |
| scenario_sim (RC, 실기) | 81      | 58.0% → 61.7% (+3.7p)  | 2.23% / 2.03% | 61.0 → 31.7%     |
| SUMO (실차)             | 24      | 45.8% → 66.7% (+20.9p) | 1.32% / 4.86% | 53.9 → 69.1%     |

쓸림 해소. RC가 규칙 아래(31.7%)에서 규칙 위(61.7%)로 올라왔고 SUMO 이득(+21p)도 유지, RC 오경보는 오히려 감소. 현준 기준("RC가 규칙 이기면 끝") 충족 → risk\_model\_streams\_82.json = 배포 후보.  
 솔직한 단서: RC +3.7p는 위험 시나리오 81개 기준이라 큰 차이는 아닙니다. 정확한 표현은 "규칙보다 나빠지지 않으면서 실차 속도를 배웠다". 실기에 결지는 오늘 대본 라벨 현장 점수로 판단하는 게 맞습니다. (b) 가중치·(c) 속도대 분리·transformer 스왑은 지시대로 미뤘습니다 — (a)로 목표 달성.

### 4. 부록 ③ — v5 위험지도 데이터에 GPS 잡음 옵션

AI\_Model/transformer/build\_dataset\_local.py에 GPS\_NOISE 플래그 추가. 좌표를 읽은 직후 distance\_m 계산 전에 gps\_noise.add\_gps\_noise(df, seed=scenario\_seed(...))를 적용해 거리/TTC/DCPA가 전부 잡음 좌표를 쓰게 함. 기본은 꺼짐(기존 동작 그대로).

```
python build_dataset_v5.py --gps-noise # → training_dataset_v5base_noisy.csv
```

검증(scenario\_9001): 깨끗한 거리 대비 오차 평균 2.50 m( $\sigma$  2.5 m 실측값과 일치), 최대 11.9 m. 라벨 분포가 상위 등급에서 흩어지는 현장 효과 재현됨 (L3 4→2, L2 60→129).

## 5. 수정·추가된 파일 (GitHub main f4df259)

모두 rsu/v2x/03\_jetson/auto\_pipeline/ 아래 (③만 AI\_Model/transformer/). 팀 기존 코드는 한 줄도 수정하지 않았습니다 — 전부 신규 파일이거나 제 스크립트 수정입니다.

| 파일                                          | 구분 | 역할                                                                                                                    |
|---------------------------------------------|----|-----------------------------------------------------------------------------------------------------------------------|
| sumo_batch_to_streams.py                    | 신규 | SUMO 시나리오 폴더 일괄 → sim_to_rsu_stream 실행. 시나리오마다 참값 거리 기준 최근접 (차량,보행자) 쌍 K개 자동 선택. 기존 기본값(첫 id)은 hit 행이 0이라 위험 표본이 없었음. |
| train_streams_model.py                      | 신규 | build_dataset_from_streams가 저장한 CSV로 train_and_export만 재실행 (아래 버그 우회). id를 str로 통일 후 팀 함수 그대로 호출.                     |
| train_alarm_transformer.py                  | 신규 | 부탁 ②. 15열-y_train-10프레임-시나리오 단위 적시경보율-규칙과 같은 오경보율-ONNX 내보내기. --csv --out --epochs.                                    |
| eval_streams_by_source.py                   | 신규 | 저장된 트리 모델을 원천별(sumo / scenario_sim)로 나눠 적시경보율 채점. 쓸림을 보려면 이걸로.                                                        |
| risk_model_streams.json                     | 산출 | 부탁 ① 트리 모델, 4:1 SUMO 우위 (배포 X — 참고용).                                                                                 |
| risk_model_streams_82.json                  | 산출 | (a) 8:2 배합 트리 모델 — 배포 후보. 임계값 0.8380 포함.                                                                              |
| models_alarm_tf/                            | 산출 | transformer ONNX + .pt + scaler JSON.                                                                                 |
| results/*.log                               | 산출 | 트리·transformer 학습 로그 원문.                                                                                              |
| 결과-SUMO투입_20260818.md                       | 문서 | 이 PDF와 같은 내용의 마크다운.                                                                                                   |
| AI_Model/transformer/build_dataset_local.py | 수정 | 부탁 ③ GPS_NOISE 옵션 (기본 꺼짐).                                                                                            |
| AI_Model/transformer/build_dataset_v5.py    | 수정 | --gps-noise 플래그.                                                                                                      |

미푸시(용량): training\_dataset\_streams.csv(988 MB), sumo\_streams/(1,401쌍). 재생성 약 1.5시간 — 필요하면 말해주면 서버에 올려둘게.

## 6. 발견한 팀 코드 버그 (안 고쳤음 — 현준이 판단)

build\_dataset\_from\_streams.py --streams ... --from-scenario-sim ... --train을 같이 주면 학습 단계 dataset.split\_by\_scenario의 sorted(ids)에서 str(sumo id) vs int(scenario\_sim id) 비교로 TypeError. 한 원천만 쓰면 안 나서 지금까지 못 만난 버그.

데이터셋 CSV는 그 전에 저장되므로 train\_streams\_model.py로 학습만 다시 돌리면 됨. 근본 수정은 build\_from\_streams\_dir/build\_from\_scenario\_sim에서 scenario\_id를 str로 통일하면 한 줄.

## 7. 재현 명령

```
cd rsu/v2x/03_jetson/auto_pipeline
python sumo_batch_to_streams.py <v5data> --out sumo_streams --pairs 2 --randomize # (1) ~40 min
python build_dataset_from_streams.py --streams sumo_streams --from-scenario-sim 4000 \
    --families --randomize --out training_dataset_streams.csv # (2) ~40 min, NO --train
python train_streams_model.py --csv training_dataset_streams.csv --model risk_model_streams.json
python eval_streams_by_source.py
python train_alarm_transformer.py --csv training_dataset_streams.csv --out models_alarm_tf --epochs
6 # (3) ~80 min
```

(2)에서 --train을 같이 주면 6장의 버그로 죽으므로 빼고, 학습은 train\_streams\_model.py로. v5data = AI\_Model/transformer/v5data/ (Jetson incoming\_data의 scenario\_9001~9700 사본, 로컬만).

### (a) 8:2 배합 재현 (배포 후보 모델)

```
python sumo_batch_to_streams.py <v5data> --out sumo_streams_small --pairs 1 --limit 90 --randomize
# 3 min
python build_dataset_from_streams.py --streams sumo_streams_small --from-scenario-sim 4000 \
    --families --randomize --out training_dataset_streams_82.csv # ~30 min
python train_streams_model.py --csv training_dataset_streams_82.csv --model
risk_model_streams_82.json
python eval_streams_by_source.py --csv training_dataset_streams_82.csv --model
risk_model_streams_82.json
```